

Image Watermarking Tool

A Deep Dive: every module, every decision, from zero prior knowledge

Portfolio explainer

The whole project in five sentences

This is a desktop application with a window, buttons and sliders. You give it one small image (a logo, called the *watermark*) and a pile of photos. You drag sliders to choose how big the logo should be, how see-through it should be, and where on the photo it should sit. The window shows you the result live, side by side with the untouched original, before anything is written to disk. When you are happy, one button stamps that same logo onto every photo you selected and saves copies, leaving your originals alone.

Contents

1	Orientation: what you are looking at	2
1.1	The size of the thing	2
1.2	The directory	2
1.3	The two dependencies, and what each is for	2
2	Architecture	3
2.1	The shape of the program	3
3	The four pure functions	4
3.1	compute_watermark_box(watermark, max_width, max_height)	4
3.2	apply_opacity(watermark, opacity_percent)	5
3.3	compute_position(...)	6
3.4	paste_watermark_at(base_image, watermark, x, y)	7
4	WatermarkApp: the window and the state	8
4.1	What the class holds	9
4.2	Building the layout	9
4.3	The event flow and the debounce	10
5	The two pipelines, and the promise between them	11
5.1	The design claim	11
5.2	Where the promise breaks	12
6	Output: save_watermarked_file	12
6.1	apply_and_save: the batch loop	13
7	Smaller findings	14
7.1	Things done right, stated plainly	14
8	Running it	15

1 Orientation: what you are looking at

1.1 The size of the thing

The entire program is one file, `main.py`, at 389 lines. It has one external dependency (Pillow) and one dependency that ships with Python itself (tkinter). There are no tests, no configuration files, no environment variables, no database, no network access. It reads image files and writes image files.

This document is proportional to that. It covers the project completely and then stops. If it seems short next to a document about a distributed database, that is correct: padding a small tool into a large document would make it a worse document.

Jargon: Desktop GUI application

GUI stands for Graphical User Interface: a program you interact with by looking at a window and clicking things, as opposed to a command-line program where you type text. “Desktop” means it runs on your own computer as its own window, rather than in a web browser or on a server somewhere.

1.2 The directory

```
image-watermarking-tool
├── main.py    the entire program, 389 lines
├── requirements.txt    one line: Pillow
├── README.md
├── LICENSE
├── .gitignore
├── docs
│   ├── screenshots
│   │   └── app.png    a picture of the running window
│   └── explainers
│       ├── deep-dive.tex    this document
│       └── brief.tex
```

Figure 1: Every tracked file in the repository. There is no `src/` folder, no package, no module hierarchy, because at 389 lines there is nothing to organise.

1.3 The two dependencies, and what each is for

Jargon: tkinter

The graphical toolkit that comes bundled with Python. It provides the window, the buttons, the sliders and the drawing surfaces. It is a Python wrapper around an older toolkit called Tk. You do not install it with `pip`; it arrives with Python (though on stripped-down Linux systems it can be a separate operating-system package, which is why the README mentions `apt install python3-tk`).

Jargon: Pillow (imported as PIL)

The image library. It is the thing that actually knows how to read a JPEG off the disk, shrink it, change how transparent it is, paste one image onto another, and write the result back out. Every pixel operation in this project is Pillow doing the work. `requirements.txt` pins it to `Pillow>=10.0,<13.0`, meaning version 10 or newer but not version 13, which is a defensive range: it accepts bug fixes but refuses a future major version that might change behaviour.

The division of labour is clean and worth stating up front, because it is the project's main organising idea:

tkinter	knows about windows, clicks and slider positions. Knows nothing about pixels.
Pillow	knows about pixels. Knows nothing about windows.

The two meet at exactly one point: `ImageTk.PhotoImage`, a bridge class that converts a Pillow image into something tkinter is willing to draw on a canvas.

2 Architecture

2.1 The shape of the program

`main.py` has two distinct halves, and understanding the split is most of understanding the project.

The first half is four **pure functions**. They take images and numbers in, return images and numbers out, and touch no window, no slider and no disk. The second half is one **class**, `WatermarkApp`, which builds the window, holds the state, listens for clicks, and calls the four pure functions when it needs pixels moved.

Jargon: Pure function

A function whose output depends only on the values you pass in, and which changes nothing outside itself. Given the same inputs it always returns the same thing. This matters here for one very practical reason: a pure function can be called from two completely different places (the live preview and the final save) and be trusted to behave identically in both. That property is the project's central design claim.

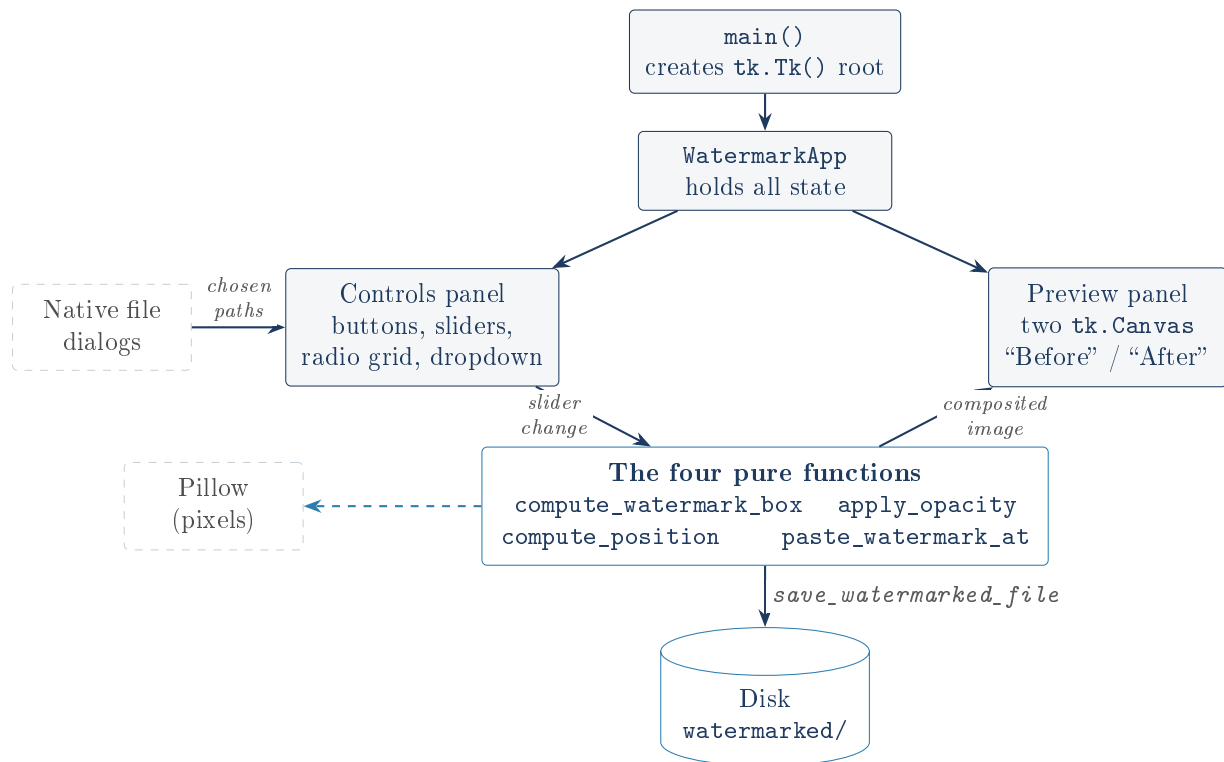


Figure 2: The architecture. Everything above the blue box is user interface; the blue box is where images are actually manipulated. Solid arrows are control and data flow; the dashed arrow marks the pure functions delegating the real pixel work to Pillow.

3 The four pure functions

These are the load-bearing part of the program. Each is short and each does exactly one thing.

3.1 `compute_watermark_box(watermark, max_width, max_height)`

```

1 def compute_watermark_box(watermark, max_width, max_height):
2     resized = watermark.copy()
3     resized.thumbnail((max_width, max_height), Image.LANCZOS)
4     return resized

```

Three lines, and two of them are doing something subtle.

`watermark.copy()` exists because Pillow’s `thumbnail()` modifies the image *in place* rather than returning a new one. Without the copy, every preview render would permanently shrink the user’s loaded watermark, and dragging a slider back up would not restore it: the pixels would be gone. The copy is what makes the sliders non-destructive.

Jargon: Bounding box vs exact size

`thumbnail(w, h)` does not resize the image to exactly `w` by `h`. It shrinks the image until it *fits inside* a `w` by `h` rectangle while keeping its shape (its aspect ratio) unchanged. A 200x100 logo asked to fit a 600x600 box would come out 200x100; asked to fit a 60x60 box it comes out 60x30. This is why the sliders are labelled “Max width” and “Max height” rather than “Width” and “Height”.

Jargon: Aspect ratio

The ratio of an image’s width to its height. Preserving it means the image is never stretched or squashed, only made uniformly bigger or smaller. A logo that ignored aspect ratio would come out visibly distorted.

Jargon: LANCZOS

The named algorithm Pillow uses to decide what colour each pixel of the shrunk image should be. When you shrink an image, several original pixels have to collapse into one, and the choice of how to average them affects quality. LANCZOS is the slowest and highest-quality of Pillow’s options, which is a defensible choice here because the images are small and the operation is infrequent.

thumbnail never enlarges, and that has consequences

This is the single most important behavioural fact in the project, and it is not stated in the README. `thumbnail()` only ever shrinks. If the watermark is 200x100 and the user drags “Max width” to 600, nothing happens: the watermark stays 200x100. The slider is a shrink-only control that silently does nothing across the whole upper part of its range for any watermark smaller than the slider value. Since the sliders default to 150 and run to 600, and logos are often small, users will meet this. Verified by running `compute_watermark_box` on a 200x100 image with a 600x600 box: the result was 200x100. Section 5 shows that this is not merely cosmetic; it is the root cause of a real correctness bug.

3.2 `apply_opacity(watermark, opacity_percent)`

```
1 def apply_opacity(watermark, opacity_percent):
2     wm = watermark.convert('RGBA')
3     r, g, b, a = wm.split()
4     scale = opacity_percent / 100.0
5     a = a.point(lambda v: int(v * scale))
6     wm.putalpha(a)
7     return wm
```

Jargon: RGBA and the alpha channel

A colour image stores, for every pixel, how much Red, Green and Blue it has: RGB. An RGBA image stores a fourth number per pixel, Alpha, which records how opaque that pixel is. Alpha runs 0 to 255, where 255 means fully solid and 0 means fully invisible. A JPEG has no alpha channel at all; a PNG can have one. Alpha is what makes a watermark look faded rather than pasted like a sticker.

The function does four things. `convert('RGBA')` guarantees there is an alpha channel to work with, inventing a fully opaque one (all 255) if the source was a JPEG. `split()` separates the image into its four channels as four separate greyscale images. `a.point(...)` runs a function over every pixel of the alpha channel, multiplying it by the requested fraction. `putalpha(a)` puts the scaled channel back.

Jargon: `.point()`

Pillow’s way of saying “apply this small function to every single pixel value”. It is implemented in C internally, so it is far faster than a Python loop over pixels would be, while reading almost like one.

Note that opacity is applied *multiplicatively*, not by assignment. A watermark PNG that was already 50% transparent in places, set to 50% opacity here, ends up at 25% in those places. This is the correct behaviour (it respects the artwork's own transparency) but it means the slider is a relative dimmer, not an absolute setting. Verified: `apply_opacity` on a fully opaque red pixel at 50 returns alpha 127, and at 0 returns alpha 0.

`apply_opacity` always returns RGBA, and this is a bigger deal than it looks

Because the first line is an unconditional `convert('RGBA')`, this function can never return anything but an RGBA image, whatever you feed it. Since both the preview path and the save path always run the watermark through it, the watermark is *always* RGBA downstream. Section 6 shows that this single fact silently kills the JPEG option in the output dropdown.

3.3 `compute_position(...)`

```
1 def compute_position(base_w, base_h, wm_w, wm_h, preset, offset_x_pct, offset_y_pct):
2     base_x, base_y = POSITION_PRESETS[preset](base_w, base_h, wm_w, wm_h)
3     x = base_x + int((offset_x_pct / 100.0) * base_w)
4     y = base_y + int((offset_y_pct / 100.0) * base_h)
5     return x, y
```

Position is computed in two stages: snap to one of nine presets, then nudge by a percentage of the photo's own dimensions.

The nine presets live in `POSITION_PRESETS`, a dictionary mapping a name to a small anonymous function:

```
1 POSITION_PRESETS = {
2     'top-left':      lambda bw, bh, ww, wh: (0, 0),
3     'center':        lambda bw, bh, ww, wh: ((bw - ww) // 2, (bh - wh) // 2),
4     'bottom-right':  lambda bw, bh, ww, wh: (bw - ww, bh - wh),
5     # ... nine in total
6 }
```

Jargon: lambda, and a dictionary of functions

A `lambda` is a function written inline without a name. Storing them in a dictionary means “look up the preset by name and run whatever formula belongs to it”, which replaces what would otherwise be a nine-branch `if/elif` chain. Adding a tenth position would be one new line rather than a new branch. This is a recognised pattern sometimes called a dispatch table.

Jargon: Image coordinates

Pixel coordinates start at (0, 0) in the *top-left* corner, with `x` growing rightwards and `y` growing *downwards*. This is upside-down compared to the graphs taught in school, and it is why ‘top-left’ is (0, 0) and ‘bottom-right’ is (bw - ww, bh - wh): to place the watermark's own top-left corner such that its bottom-right corner lands on the photo's bottom-right corner, you subtract the watermark's size.

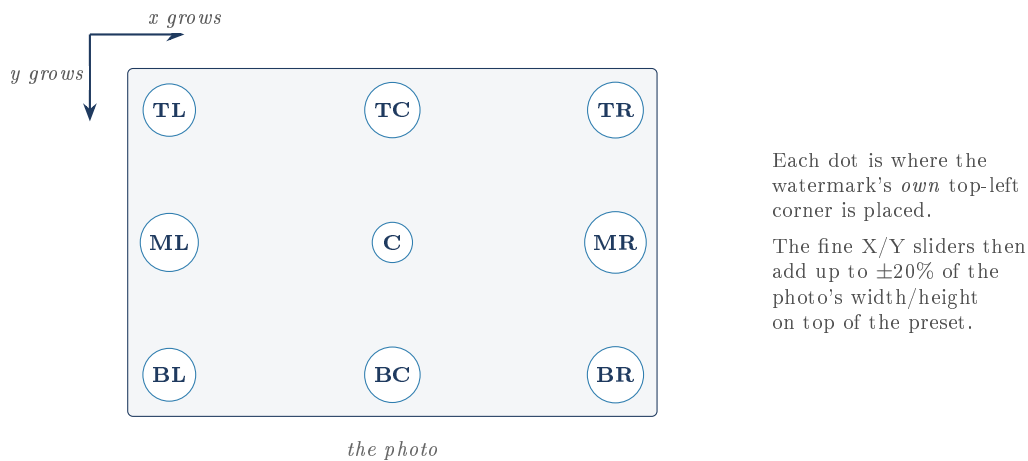


Figure 3: The 3x3 preset grid, exposed in the window as nine radio buttons labelled TL, TC, TR, ML, C, MR, BL, BC, BR (from `POSITION_LABELS`).

The offsets are expressed as a percentage *of the photo*, not of the watermark and not in pixels. This is the right choice: it means the same settings produce a visually equivalent placement on a 600x400 photo and on a 6000x4000 photo, which is exactly what batch mode needs.

Nothing clamps the position to the canvas

`compute_position` can return coordinates entirely outside the photo, and nothing checks. Verified: a 3000x2000 photo with a 200x100 watermark at `bottom-right` with both offsets at +20 returns (3400, 2300), which is well past the bottom-right corner. Pillow does not error on an off-canvas paste; it simply clips, so the watermark vanishes entirely. The save then still reports “Saved 1 watermarked photo(s).” The user gets a silent no-op presented as success. The preview would show the same emptiness, which is the one saving grace, so a watching user would notice. Clamping `x` to `[0, base_w - wm_w]` would be a two-line fix.

3.4 `paste_watermark_at(base_image, watermark, x, y)`

This function is the heart of the project and the subject of its main war story.

```
1 def paste_watermark_at(base_image, watermark, x, y):
2     result = base_image.convert('RGBA') if watermark.mode == 'RGBA' else base_image.copy()
3     if watermark.mode == 'RGBA':
4         result.paste(watermark, (x, y), watermark)
5     else:
6         result.paste(watermark, (x, y))
7     return result
```

Jargon: Alpha compositing

The act of laying a partly transparent image over another and computing, for each pixel, the correct blended colour: mostly the top image where it is opaque, mostly the bottom image where it is transparent, and a weighted mix in between. It is what “fading a logo onto a photo” means mathematically.

The bug this function exists to fix

Pillow’s `paste(image, (x, y))` does **not** blend. With only two arguments it *overwrites* the destination pixels outright, copying the watermark’s RGB *and* its alpha straight over whatever was underneath. The photo’s pixels in that rectangle are simply discarded.

The trap is that this still *looks* plausible. The faded alpha value does get written into the output file, so any image viewer that respects alpha renders the watermark as translucent, and the result passes a glance. But it is translucent against nothing: the photo underneath it was thrown away, not mixed in.

The fix is the third argument. `paste(watermark, (x, y), watermark)` passes the watermark as its own *mask*, which tells Pillow to use the watermark's alpha channel as the recipe for blending rather than as data to copy.

3.4.1 Proving it, rather than asserting it

The README claims specific pixel values as evidence. Those numbers were re-derived independently for this document by running both code paths: a solid blue (0,0,255) photo, a solid red (255,0,0) watermark faded to 50%, and the pixel read at the same covered location under each.

Code path	Pixel at the covered spot	Verdict
<code>paste(wm, (x,y))</code>	(255, 0, 0, 127)	flat red; blue is <i>zero</i>
<code>paste(wm, (x,y), wm)</code>	(127, 0, 128, 191)	genuine red/blue mix

Both reproduced exactly. The reasoning is airtight and worth being able to say out loud: in the broken path the blue channel is **0**, and the base image was pure blue. If any blending had occurred, blue could not possibly be zero. Meanwhile red sits at its full 255, untouched by the 50% fade. In the fixed path blue has climbed to 128 and red has fallen to 127, which is the weighted average you would predict from a half-opaque red over a solid blue.

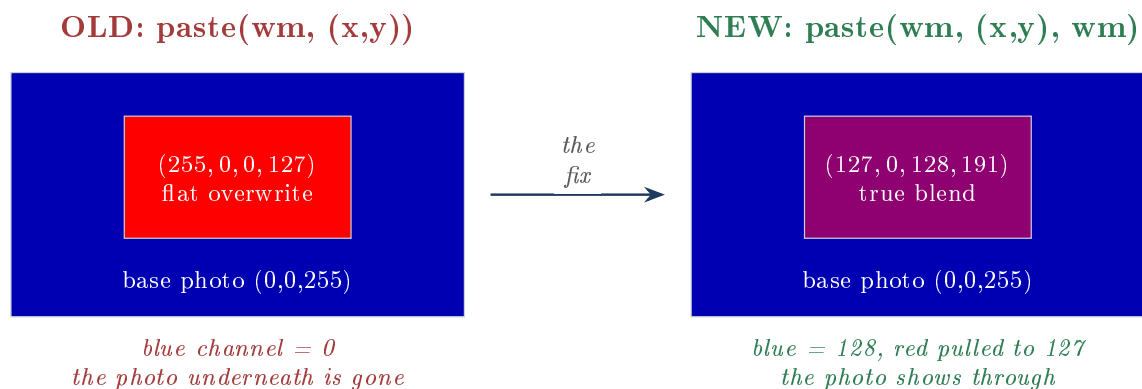


Figure 4: The alpha-compositing bug, drawn. Both versions produce a file whose alpha channel says “translucent”. Only the right-hand one actually mixed in the photo.

Half of this function is unreachable

The `else` branches exist for a non-RGBA watermark, but no such watermark ever arrives. Every caller (both the preview and the save path) runs the watermark through `apply_opacity` first, which unconditionally converts to RGBA. So `watermark.mode` is always `'RGBA'`, the ternary always picks `convert('RGBA')`, and the `else` branch is dead code. It is harmless and arguably good defensive hygiene for a function that presents itself as general purpose, but it is dead, and an interviewer who traces the callers will find it.

4.1 What the class holds

`WatermarkApp.__init__` sets up the window (980x620, with a 860x560 minimum) and then declares every piece of state the program has:

Attribute	Type	Holds
<code>watermark_original</code>	Image or None	the raw logo, never modified
<code>photo_paths</code>	tuple of paths	the selected batch
<code>_preview_after_id</code>	handle or None	the pending debounce timer
<code>_before_imgtk</code>	PhotoImage	anti-garbage-collection anchor
<code>_after_imgtk</code>	PhotoImage	anti-garbage-collection anchor
<code>width_var, height_var</code>	IntVar (150, 150)	slider values
<code>opacity_var</code>	IntVar (50)	slider value
<code>position_var</code>	StringVar	preset, def. bottom-right
<code>offset_x_var, offset_y_var</code>	IntVar (0, 0)	fine nudges
<code>format_var</code>	StringVar	format, def. JPEG
<code>location_var</code>	StringVar (empty)	custom save folder
<code>watermark_label_var</code>	StringVar	on-screen text
<code>photos_label_var</code>	StringVar	on-screen text
<code>status_var</code>	StringVar	on-screen text

Jargon: Tk variables (IntVar, StringVar)

Ordinary Python variables cannot notify anyone when they change. Tk variables are special wrapper objects that can: you attach one to a slider and to a label, and when the slider moves, the variable updates and the label follows automatically. They are read with `.get()` and written with `.set()`, never by plain assignment. They are the glue that keeps the widgets and the state in sync without manual bookkeeping.

Why `watermark_original` is never touched

The uploaded watermark is stored once and treated as read-only forever. Every preview and every save builds a fresh working copy from it, resizes that copy, fades that copy, and throws it away. This is what makes every slider non-destructive and freely reversible in both directions. The alternative (mutating a single working watermark as the sliders move) would mean opacity 50 followed by opacity 100 leaves you at 50% of 50%, permanently. The README notes this is a deliberate improvement over an earlier version that used module-level globals.

Jargon: Garbage collection, and why `_before_imgtk` exists

Python automatically frees objects nothing refers to any more. `ImageTk.PhotoImage` has a notorious interaction with this: tkinter's canvas holds only a weak internal handle to the image, so if the only Python reference goes out of scope at the end of the function, Python frees it and the image silently disappears from the window, leaving a blank rectangle and no error. Storing it on `self` keeps a live reference alive for as long as the app exists. The comment in the source, “keep references so Tk doesn't GC them”, is guarding against exactly this.

4.2 Building the layout

`_build_layout` splits the window into a left controls column and a right preview area, with a status label pinned along the bottom. `_build_controls` then fills the left column with four numbered `LabelFrame` groups, which is a small but real usability decision: the interface reads as a sequence

(1. Load images, 2. Size and opacity, 3. Position, 4. Output) rather than as an undifferentiated wall of sliders.

Jargon: Geometry managers: pack vs grid

tkinter offers two ways to arrange widgets inside a container. `pack` stacks them along an edge; `grid` places them in rows and columns. The rule that bites people is that you may not use both inside the *same* container: doing so hangs or errors. This code obeys it, using `pack` for the coarse column layout and `grid` inside the frames that need alignment, and giving the offset sliders their own sub-frame precisely so the two never mix in one parent. The README records that this exact clash was a real bug caught during development.

`_add_slider` is a small factory that builds a label, a `ttk.Scale` and a live numeric readout in one call, wiring the readout via `var.trace_add('write', refresh)` so it follows the variable automatically.

The status label is always red

`status` is created with `fg='#b00020'`, a red normally reserved for errors, and that colour never changes. The success message “Saved 5 watermarked photo(s).” therefore renders in the same alarming red as “Upload a watermark first.” A one-line fix (set the colour alongside the text) would remove a real moment of user confusion. Minor, but it is the kind of detail worth having noticed before someone else points at the screenshot.

4.3 The event flow and the debounce

```
1 def _schedule_preview(self, *_):
2     if self._preview_after_id is not None:
3         self.root.after_cancel(self._preview_after_id)
4     self._preview_after_id = self.root.after(60, self._render_preview)
```

Jargon: The event loop, and `after()`

A GUI program spends its life in a loop (started by `root.mainloop()`) waiting for things to happen: a click, a keypress, a slider drag. `root.after(60, fn)` means “run `fn` once, 60 milliseconds from now”, and hands back a handle you can use to cancel it. Crucially this is not a background thread; the callback runs on the same single thread when the loop next gets a moment, which is why nothing here needs locks.

Jargon: Debouncing

Dragging a slider fires a change event for every pixel of movement, potentially dozens per second. Redrawing on each one would queue up far more work than the program can do, and the window would lag behind the mouse. Debouncing means: whenever an event arrives, cancel the previously scheduled work and reschedule it a moment later. The result is that a burst of rapid events collapses into exactly one render, fired 60ms after the user’s last movement. 60ms is below the threshold where a human perceives delay, so it feels instant while doing a fraction of the work.

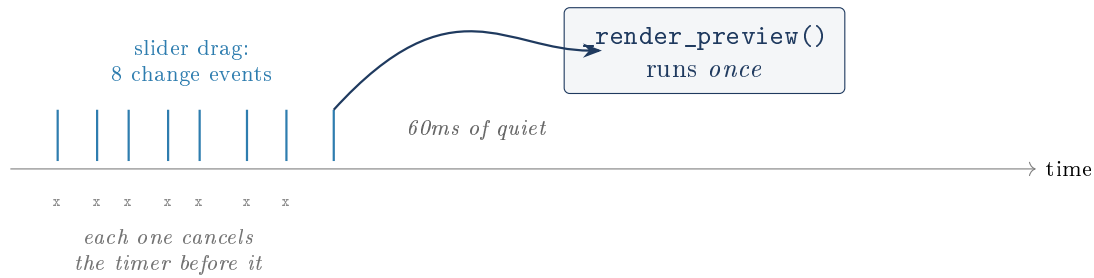


Figure 5: Debouncing. Seven of the eight events do no work at all: they exist only to cancel the timer their predecessor set. Only the final one survives its 60ms and renders.

The debounce is bypassed on upload

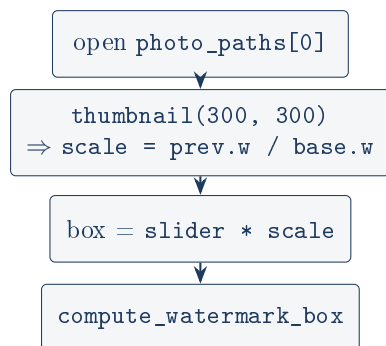
`__init__` calls `_schedule_preview()` directly, as do `upload_watermark` and `upload_photos`. That is correct and harmless. But note that `_render_preview` reopens the base photo *from disk* on every single tick (`Image.open(self.photo_paths[0])`), and no cache exists. At 60ms debounce and interactive speeds this is survivable, and the README honestly flags it under “What I’d do differently”. On a 50-megapixel source photo, though, a slider drag becomes a sustained re-decode loop, and the debounce is the only thing standing between the user and a visibly stuttering window. Caching the opened image would cost about three lines.

5 The two pipelines, and the promise between them

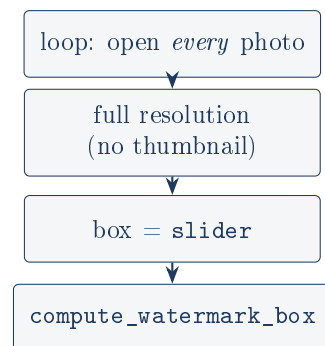
5.1 The design claim

The project’s headline claim, stated in the README, is that the preview and the save call the same compositing function, so “what’s shown on screen before saving is what actually gets written to disk, not an approximation”. This is the most interesting thing about the project and deserves careful checking.

PREVIEW path (`_render_preview`)



SAVE path (`apply_and_save`)



*the ONLY divergence:
the requested box is scaled
on the left, raw on the right*

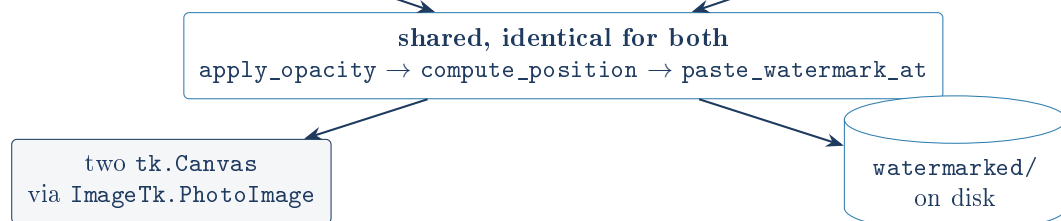


Figure 6: Preview and save. The bottom half is genuinely shared code, which is why the colour, the fade and the placement are trustworthy. The top half differs, and that is where the guarantee leaks.

The claim is largely true, and the design is sound. `apply_opacity`, `compute_position` and `paste_watermark_at` are called identically from both paths, so opacity and placement really are WYSIWYG. Placement in particular is safe by construction, because the offsets are percentages of the base image, and the preview's base is a proportional thumbnail of the real one.

Jargon: WYSIWYG

“What You See Is What You Get”. A promise that the on-screen representation matches the final output exactly, rather than approximating it.

5.2 Where the promise breaks

The preview asks for a watermark box of `slider * scale`; the save asks for `slider`. That scaling is correct in intent. But it collides with the fact established earlier that `thumbnail` refuses to enlarge, and the collision produces a real bug.

Consider a 3000x2000 photo, a 200x100 logo, and the width slider at 600. The preview thumbnails the photo to 300x200, so `scale` is 0.1 and it asks for a 60x60 box: 60 is smaller than the logo's 200, so the logo *does* shrink, to 60x30, which is 20% of the preview's width. The save asks for a 600x600 box: 600 is larger than 200, so `thumbnail` does nothing at all and the logo stays 200x100, which is 6.7% of the photo's width.

Scenario (photo, logo, slider)	Preview	Saved	
3000x2000, 800x400 logo, slider 600	20.0%	20.0%	agree
3000x2000, 200x100 logo, slider 600	20.0%	6.7%	diverge
3000x2000, 200x100 logo, slider 150	5.0%	5.0%	agree
600x400, 200x100 logo, slider 150	25.0%	25.0%	agree

The WYSIWYG guarantee fails when the slider exceeds the watermark's native size

Measured, not reasoned: the figures above come from running both paths and comparing the watermark's width as a fraction of its canvas. The preview shows a logo three times larger than the one that gets saved.

The condition is precise. Divergence occurs when `slider * scale` is below the watermark's native width but `slider` is above it, because then the preview's request shrinks the logo and the save's request does not. When the watermark is natively larger than the slider value, both paths clamp proportionally and the preview is honest. Since the sliders reach 600 and many logos are smaller than that, the broken region is reachable in ordinary use, and it gets *easier* to hit the larger the source photo is (a bigger photo means a smaller `scale`, which pushes the preview's request further down).

The root cause is not the scaling; it is that `thumbnail` is an asymmetric operation being used as if it were symmetric. A fix would either clamp both paths to `min(slider, native_size)` before scaling, or replace `thumbnail` with an explicit aspect-preserving `resize` that is willing to enlarge (which would also make the sliders do what their labels promise). This is the most defensible bug in the project to talk about: the design instinct (share the compositing function) was right, and the leak came in through a Pillow API asymmetry one layer above it.

6 Output: `save_watermarked_file`

```

1 def save_watermarked_file(image, source_path, location, file_format):
2     source = Path(source_path)
3     out_dir = Path(location) if location else source.parent / 'watermarked'
```

```

4     out_dir.mkdir(parents=True, exist_ok=True)
5
6     suffix = 'png' if image.mode == 'RGBA' else file_format.lower()
7     out_path = out_dir / f'{source.stem}_watermarked.{suffix}'
8
9     if image.mode == 'RGBA':
10         image.save(out_path, 'PNG')
11     else:
12         image.save(out_path, file_format)

```

Jargon: `pathlib.Path`

The modern way to handle file paths in Python. `source.parent` gives the containing folder, `source.stem` gives the filename without its extension, and the `/` operator joins path pieces. It matters because it uses the right separator per operating system automatically. The README notes the original code split paths on `'/'` by hand, which breaks on Windows.

`mkdir(parents=True, exist_ok=True)` creates the destination folder including any missing intermediate folders, and does not complain if it already exists. Writing to a `watermarked/` subfolder rather than beside the source is a good default: the originals are never touched, and a second run does not have to hunt for its own previous output.

The JPEG option in the format dropdown can never be selected, ever

Follow the modes. `apply_opacity` always returns RGBA. `paste_watermark_at` sees an RGBA watermark and therefore always returns an RGBA result. So the `image.mode == 'RGBA'` test here is always true, `suffix` is always `'png'`, and the file is always saved as PNG. The `file_format` parameter is read on two lines that never execute.

Verified end to end: a save with the format explicitly set to `'JPEG'` produced `photo_watermarked.png`. The dropdown at the bottom of the window offers a choice between JPEG and PNG, defaults to JPEG, and has no effect whatsoever on any possible run of the program.

The README does flag related behaviour, saying “RGBA output still forces PNG regardless of the user’s format choice”, which is honest as far as it goes. But it frames this as a conditional edge case, when tracing the callers shows it is unconditional. The dropdown is not occasionally overridden; it is inert. That is the sharper and more defensible way to state it, and it changes the remedy: the honest options are to delete the control, or to flatten the RGBA result onto a white background when the user asks for JPEG (`Image.alpha_composite` onto an opaque canvas, then `convert('RGB')`), which is what the control implicitly promises.

6.1 `apply_and_save`: the batch loop

The loop opens each photo, rebuilds the watermark *for that photo’s actual dimensions*, composites, saves, and counts. Rebuilding per photo is correct: it is what allows a mixed batch of portrait and landscape photos at different resolutions to receive visually consistent placement, because the percentage offsets resolve against each photo’s own size.

Failures are counted as if they never happened

```

1  try:
2      base = Image.open(path); base.load()
3  except Exception:
4      continue

```

A photo that cannot be opened (corrupt, deleted between selection and save, wrong permis-

sions) is skipped silently. It is not counted in `saved`, so the total is accurate, but the user is told “Saved 9 watermarked photo(s).” after selecting ten and is never told which one failed or that anything failed at all. A bare `except Exception` around both the open and the load also swallows genuinely unexpected errors. Collecting the failures and reporting “Saved 9, skipped 1 (bad.jpg)” would be a handful of lines.

The same pattern appears in `upload_watermark` and `_render_preview`, though there it is better behaved: both set a specific message into `status_var` or draw one onto the canvas, so the user does learn something went wrong.

7 Smaller findings

Dead import

Line 8 is `import os`. Verified by `grep`: the string `os.` appears nowhere in the file. The module is imported and never used, a leftover from the pre-`pathlib` version the README describes. One line to delete.

No tests at all

There is no test file and no test framework in `requirements.txt`. Every claim in the README rests on ad hoc verification scripts pasted into the README itself rather than anything that can be re-run by `pytest` on a change. The README names this as the first thing it would do differently, which is the right call: the four pure functions are perfectly testable (they take images and numbers and return images and numbers, with no window in sight), and the two bugs documented in this section 5 and section 6 would each have been caught by a three-line assertion. This is the clearest gap in the project.

The README’s “Challenges” section documents a LaTeX problem

One of the five bullets under “Challenges” is about a TikZ style name colliding with a `pgfplots` key while building these explainer PDFs. That is a real problem that really happened, but it is a problem with the documentation toolchain, not with the image watermarking tool, and it sits oddly in a list otherwise about alpha compositing and preview performance. A reader skimming for engineering substance may read it as padding. It would sit better in the portfolio’s own notes.

7.1 Things done right, stated plainly

Honesty runs both ways, and the following are genuine strengths worth defending:

- The pure-function split is real and it is the reason the compositing is trustworthy across two call sites. Most small GUI tools of this kind smear the pixel logic through the button handlers.
- The `watermark_original` read-only discipline makes every control non-destructive without any undo machinery.
- The alpha-compositing bug was found, fixed, and then *proved* with a synthetic-pixel experiment rather than by eyeballing the output. Given that the broken version looks correct in most viewers, eyeballing would have missed it. That is the single best thing in this project.
- Percentage-based offsets are the right abstraction for a batch tool.
- The debounce, the thumbnailed preview and the reuse of the compositing function show the author thinking about the interaction between correctness and responsiveness rather than only

one of them.

8 Running it

```
1 python3 -m venv venv
2 source venv/bin/activate      # Windows: venv\Scripts\activate
3 pip install -r requirements.txt
4 python3 main.py
```

The window opens immediately with both preview panes showing “Upload a photo to preview”, because `__init__` schedules a render before anything is loaded and `_render_preview` handles the empty case first.

What is and is not verified

The pixel-level claims in this document were re-derived by executing the project’s own functions under Python 3.12.3 and Pillow 10.1.0. The alpha table, the mode tracing, the `thumbnail` non-enlargement, the WYSIWYG divergence table, the off-canvas coordinates and the JPEG-to-PNG result are all measured.

Not verified here: the interactive flow through the native file dialogs (they block on user input and cannot be driven headlessly, as the README states), and any behaviour on Windows or macOS. Everything said about the file dialogs and about cross-platform path handling is reasoned from the source, and is **inferred** rather than observed.